IC-253 (DSA) Assignment 2 Report

Aryan Sisodiya

March 11, 2026

1 Task 1: Gaming Leaderboard using Skip List

1.1 Problem Description

In this task, a real-time gaming leaderboard system is implemented using Skip List data structure. The system supports operations such as insertion, updating scores, deletion of players, searching, retrieving ranks, displaying the top players, displaying Leaderboard and visualizing the skip list structure.

1.2 Approach

Each node in the skip list stores:

- Player data (ID, Name, Score, Timestamp)
- An array of forward pointers corresponding to different levels
- The level of the node

The level of a new Node is randomly generated to probabilistically balance the structure. Searching, insertion, and deletion are performed by traversing from the highest level forward until the optimal insertion point occurs or the end of list is achieved, moving down to next level and repeating this until we get to the lowest level.

The leaderboard ordering follows:

1. Higher score ranks higher.
2. For equal scores, the earlier timestamp ranks higher.

This design ensures efficient expected time complexity for core operations.

1.3 Functions and Implementation

insertPlayer()
Inserts a new player into the skip list. The algorithm traverses the skip list from the highest level downward to determine the correct insertion point, storing the predecessor node pointers in an update array (which is of constant size) and updates forward pointers accordingly.

searchPlayer()
Searches for a player using score and timestamp by traversing levels of the skip list. This allows logarithmic expected search time.

deletePlayer()
Removes a player node by first identifying its predecessor nodes at each level and then reconnecting the forward pointers.

updateScore()
Updates a player's score by deleting the existing node and reinserting a modified player record.

getRank()
Determines the rank of a player by traversing the base level of the skip list.

getTopK()
Displays the top k ranked players from the leaderboard.

displayLeaderboard()
Prints the leaderboard in ranked order.

displaySkipListStructure()
Displays the structure of the skip list across all levels.

1.4 Time and Space Complexity Analysis

Skip Lists provide expected logarithmic complexity for search, insertion, and deletion operations due to their probabilistic balancing mechanism. The structure maintains multiple layers of linked lists that enable fast traversal across elements. Operations that require scanning the base list, such as rank computation, take linear time. The expected (Average Case) Time Complexity of various operations implemented are as follow:

Operation	Time Complexity	Space Complexity
insertPlayer()	$O(\log n)$	$O(1)$
searchPlayer()	$O(\log n)$	$O(1)$
deletePlayer()	$O(\log n)$	$O(1)$
updateScore()	$O(\log n)$	$O(1)$
getRank()	$O(n)$	$O(1)$
getTopK(k)	$O(k)$	$O(1)$
displayLeaderboard()	$O(n)$	$O(1)$
displaySkipListStructure()	$O(n)$	$O(1)$

1.5 Why is Space Complexity of Insertion and Deletion operation $O(1)$?

Although the insertion and deletion algorithms use an auxiliary array `update[]` to store the predecessor nodes at each level during traversal, the space complexity of these operations is still considered $O(1)$.

The size of the `update[]` array depends only on the maximum number of levels allowed in the skip list, denoted as `MAX_LEVEL`. In the implementation, this value is fixed at 10. Since this array size does not depend on the number of players n stored in the skip list, it remains constant regardless of how large the skip list grows. Therefore, the auxiliary space used by insertion, deletion, and update operations remains constant:

$$\text{Auxiliary Space} = O(\text{MAX_LEVEL}) = O(1)$$

Choice of Maximum Levels

In a skip list, the expected height grows logarithmically with the number of elements, approximately $\log_2(n)$ when the promotion probability is 0.5. Setting `MAX_LEVEL = 10` is sufficient for typical leaderboard sizes since $\log_2(1024) = 10$. Thus, the structure can efficiently support thousands of players while maintaining logarithmic search performance, while also keeping auxiliary structures like the `update[]` array statically bounded in size.

1.6 Test Case Demonstrations

```
Inserting Players

Leaderboard after insertion:
1 105 Farhan 990 25
2 101 Aryan 950 10
3 103 Daksh 920 30
4 104 Pranjal 870 15
5 102 Aditya 870 20

Skip List Structure:
Level 2:950 920 870
Level 1:990 950 920 870
Level 0:990 950 920 870 870

Searching Player with score 920 and time 30:
Player Found. ID = 103 Name= Daksh

Rank of Player 103: 3

Updating score of Player p2 (id 102) to 980

Leaderboard after score update:
1 105 Farhan 990 25
2 102 Aditya 980 20
3 101 Aryan 950 10
4 103 Daksh 920 30
5 104 Pranjal 870 15
```

(a) Output Part 1

```
Skip List Structure:
Level 2:950 920
Level 1:990 950 920
Level 0:990 980 950 920 870

Deleting Player p4 (id 104)

Leaderboard after deletion:
1 105 Farhan 990 25
2 102 Aditya 980 20
3 101 Aryan 950 10
4 103 Daksh 920 30

Top 3 Players:
105 Farhan 990
102 Aditya 980
101 Aryan 950

Final Skip List Structure:
Level 2:950 920
Level 1:990 950 920
Level 0:990 980 950 920
```

(b) Output Part 2

Figure 1: Task 1 – Test Case 1 Output

```
Inserting Players

Leaderboard after insertion:
1 205 Jadeja 970 15
2 203 Ishaan 910 10
3 202 Dhoni 910 18
4 201 Virat 880 12
5 204 Axar 840 25

Skip List Structure:
Level 1:970
Level 0:970 910 910 880 840

Searching Player with score 910 and time 10:
Player Found. ID = 203 Name= Ishaan

Rank of Player 203: 2

Updating score of Player q1 (id 201) to 990

Leaderboard after score update:
1 201 Virat 990 12
2 205 Jadeja 970 15
3 203 Ishaan 910 10
4 202 Dhoni 910 18
5 204 Axar 840 25
```

(a) Output Part 1

```
Skip List Structure:
Level 1:990 970
Level 0:990 970 910 910 840

Deleting Player q4 (id 204)

Leaderboard after deletion:
1 201 Virat 990 12
2 205 Jadeja 970 15
3 203 Ishaan 910 10
4 202 Dhoni 910 18

Top 3 Players:
201 Virat 990
205 Jadeja 970
203 Ishaan 910

Final Skip List Structure:
Level 1:990 970
Level 0:990 970 910 910
```

(b) Output Part 2

Figure 2: Task 1 – Test Case 2 Output

2 Task 2: Time-Bound Message Deletion using Doubly Linked List

2.1 Problem Description

This task implements a messaging system where users can delete messages either for themselves or for everyone within a limited time window of 120 seconds after sending. The system maintains chat history and supports undoing the last deletion operation.

2.2 Approach

The chat system is implemented using a Doubly Linked List where each node represents a message.

Each message node stores:

- Message ID
- Sender ID
- Message text
- Timestamp
- Deletion flags (isDeleted and isDeletedForEveryone)
- Pointers to previous and next messages

The head pointer points to the oldest message, while tail points to the latest message sent.

Messages are thus inserted sequentially at the tail of the list to maintain chronological order.

Two deletion mechanisms are implemented:

- **Delete for Me:** hides the message only from the current user's view.
- **Delete for Everyone:** replaces the message text with a placeholder if the time constraint is satisfied.

To support undo functionality, a stack is implemented using a singly linked list. Whenever a deletion occurs, the affected message node is pushed onto the stack so that we have the record of the most recent delete and the operation can be reverted later.

2.3 Functions and Implementation

sendMessage()
Creates a new message node and inserts it at the tail of the doubly linked list.

deleteForEveryone()
Marks a message as deleted for everyone if the time constraint of 120 seconds is satisfied.

deleteForMe()
Marks a message as deleted only for the current user.

undoLastDelete()
Restores the most recent deletion by popping the corresponding message from the stack and resetting its deletion flag.

displayChat()
Traverses the doubly linked list and prints the chat history, skipping the *Deleted for me* messages completely and showing *Deleted for everyone* message as "This message was deleted"

Stack Operations:

- **push():** Push the message Node into stack
- **pop():** Pop from the stack, also returns the popped message Node
- **isEmpty():** Checks whether the stack is empty or not

2.4 Time and Space Complexity Analysis

All operations involve traversing the doubly linked list or manipulating stack nodes. Since messages are stored sequentially, locating a specific message requires linear traversal. Stack operations are constant time. The time complexity for each operation implemented are as follow:

Operation	Time Complexity	Space Complexity
sendMessage()	$O(1)$	$O(1)$
findMsg()	$O(n)$	$O(1)$
deleteForEveryone()	$O(n)$	$O(1)$
deleteForMe()	$O(n)$	$O(1)$
undoLastDelete()	$O(1)$	$O(1)$
displayChat()	$O(n)$	$O(1)$
stack.push()	$O(1)$	$O(1)$
stack.pop()	$O(1)$	$O(1)$
stack.isEmpty()	$O(1)$	$O(1)$

2.5 Test Case Demonstrations

```
Sending Messages
101 Hello
102 Hi
103 How are you?

Delete Message 2 for everyone
101 Hello
This message was deleted
103 How are you?

Delete Message 3 for me
101 Hello
This message was deleted
```

(a) Output Part 1

```
Undo last delete
101 Hello
This message was deleted
103 How are you?

Undo again
101 Hello
102 Hi
103 How are you?

Attempt delete after time limit
101 Hello
102 Hi
103 How are you?
```

(b) Output Part 2

Figure 3: Task 2 – Test Case 1 Output

```
Sending Messages

Initial Chat
101 Hello everyone
102 Hi!
103 How are you all?
104 I'm good
101 Nice to hear that
102 Working on assignment

Delete message 3 for everyone
101 Hello everyone
102 Hi!
This message was deleted
104 I'm good
101 Nice to hear that
102 Working on assignment

Delete message 5 for me
101 Hello everyone
102 Hi!
This message was deleted
104 I'm good
102 Working on assignment
```

(a) Output Part 1

```
Undo last delete
101 Hello everyone
102 Hi!
This message was deleted
104 I'm good
101 Nice to hear that
102 Working on assignment

Attempt delete after time limit (message 1)
101 Hello everyone
102 Hi!
This message was deleted
104 I'm good
101 Nice to hear that
102 Working on assignment
```

(b) Output Part 2

Figure 4: Task 2 – Test Case 2 Output